COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE	**SECTION:**	CPE21S1
DATE PERFORMED:	10/6/2026	**DATE SUBMITTED:**	10/8/2026
NAME:	Nichollo Dave A. Galvez Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. # **Name**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Measure the runtime of algorithms using theoretical analysis

ANSWER

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

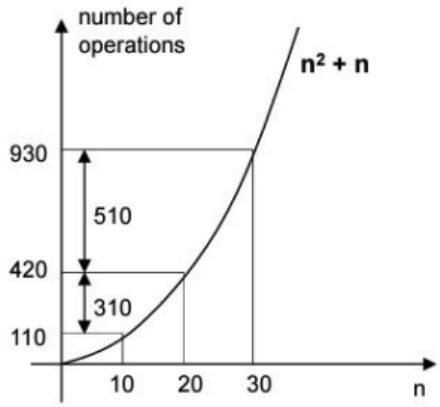
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **m × n**

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
The upward graph presents a quadratic function based on the time complexity classes. Because the size of the array is exponential.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

ANSWER

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	**0 µs**	`Time taken to search = 0 microseconds` `Student (506 303) needs vaccination.`	Our student is "506,303" and they did not appear on the list of 1000 students. So the program concluded that the student needs vaccination
10,000	**1 µs**	`Time taken to search = 1 microseconds` `Student (506 303) needs vaccination.`	We're checking for the same student but now in a list of 10,000. The time it took to look over the entire list was 1 microsecond. Since they are still

			not in the list, therefore they need vaccination.
100,000	**2 µs**	`Time taken to search = 2 microseconds` `Student (506 303) needs vaccination.`	Lastly, we are checking the same exact student but now in a list of 100,000. This took an extra 2 microseconds compared to the 1,000 list. Since they are not in the list either, they need the vaccination.

CODE

header file

```
#ifndef STUDENT_H
#define STUDENT_H

#include <algorithm>
#include <chrono>
#include <cmath>
#include <iostream>
#include <numeric>
#include <random>
#include <vector>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters
    auto get_name() const { return name; }
    auto is_vaccinated() const { return vaccinated; }

    // Two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};
```

```cpp
inline auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // The IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;

    return Student(random_name, is_vaccinated);
}

inline bool needs_vaccination(const Student& P, const std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = static_cast<long long>(std::floor(range_length / 2));
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}
```

```
inline void search_test(int size, const Student& p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") needs vaccination." << std::endl;
    } else {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") does not need vaccination." << std::endl;
    }
}

#endif
```

main cpp file

```
#include <iostream>
#include "studentSearch.h"

int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
          return false
    return true
```

THEORETICAL ANALYSIS

The algorithm evaluates array elements pairwise via nested loops, yielding a worst-case total of $\frac{n^2-n}{2}$ comparisons and a quadratic time complexity of $O(n^2)$. TheThe algorithm evaluates unique pairs as a combination $\binom{n}{2}$ by using nested loops with $j = i + 1$, avoiding redundant pair checks and resulting in $\frac{n^2-n}{2}$ total comparisons with an $O(n^2)$ worst-case time complexity.

EXPERIMENTAL ANALYSIS

The experiment successfully ran and is logically correct. In our code, we generated a random array of 10 numbers that ranged from 1 to 100. Each run, it will generate a random set of elements. We observed that as we add more members, the run time increases. But there's a catch, if the array size exceeds the number generation range it will most likely be not a unique array. And if the number generated range is higher than the size, it will most likely be a unique array.

ANALYSIS AND COMPARISON

While both analyses focus on the same algorithm, they have different approaches but common perceptions. In the theoretical analysis, it stated that the worst case scenario is $\frac{n^2-n}{2}$, the code itself hits this time complexity when the range of numbers generated is larger than N, this allows all elements to be unique. The theoretical perspective tells us the maximum possible comparisons, while the experimental perspective actually proves that data distribution and the range are important factors in getting the maximum as well.

CODE

```cpp
#include <iostream>
#include <random>
#include <chrono>
bool unique(int arr[], size_t n);

int main() {
    const size_t n = 10;
    int arr[n];

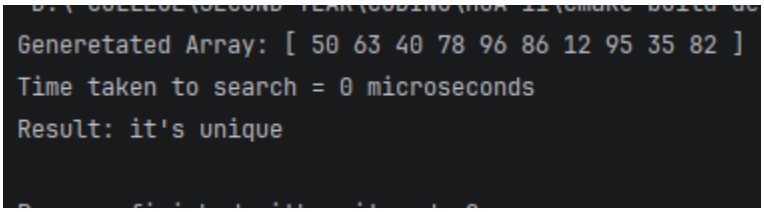
    std::random_device rd;
    std::mt19937 gen( sd: rd());
    std::uniform_int_distribution<int> dist( a: 1,  b: 100);
    std::cout << "Generetated Array: [ ";
    for (size_t i = 0; i < n; ++i) {
        arr[i] = dist( &: gen);
        std::cout << arr[i] << " ";
    }
    std::cout << "]" << std::endl;

    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool result = unique(arr, n);
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>( d: end - begin).count()
              << " microseconds" << std::endl;

    if (result) {
        std::cout << "Result: it's unique\n";
    } else {
        std::cout << "Result: it's not unique\n";
    }
    return 0;
}
bool unique(int arr[], size_t n) {
    for (size_t i = 0; i < n; ++i) {
        for (size_t j = i + 1; j < n; ++j) {
            if (arr[i] == arr[j]) {
                return false;
            }
        }
    }
    return true;
}
```

OUTPUT



Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3 y = brpower(x, (n-1)/2)
4 return x*y*y
5 if n is even then
6 y = brpower(x, n/2)
7 return y*y

THEORETICAL ANALYSIS

The first algorithm calculates x^n by calling the function, or through recursive. The time complexity of this function is O(n). The second algorithm calculates x^n by halving n at each step, making at most $2\log_2 n$ multiplications by reusing intermediate results, which yields an $O(\log n)$ logarithmic time complexity.

EXPERIMENTAL ANALYSIS

After making the C++ code from the algorithm, we can see the difference of computation time of the rpower and brpower function. Since the “rpower” function only recursively calls, this takes more time than the “brpower” function that halves the process each call. A phenomenon starts to happen when we go on to higher values. The computation time for “rpower” when we increase the values also increases, but for “brpower” the higher the values the lower the computation time.

ANALYSIS AND COMPARISON

While both analyses focus on the exponentiation algorithms, they have different perceptions. In the theoretical analysis, it stated that rpower runs in O(n) time while brpower runs in O(log n) time. The code itself demonstrates this, as computing 4^9 took 1437 microseconds for rpower versus 417 microseconds for brpower. The theoretical perspective shows the step reduction from halving n, while the experimental perspective proves that fewer multiplications and recursive calls lead to faster execution times.

CODE

```cpp
#include <iostream>
#include <chrono>

int rpower(int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

int brpower(int x, int n) {
    if (n == 0) return 1;

    if (n % 2 == 1) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }
    else {
        int y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    int x, n;

    std::cout << "Enter base x: ";
    std::cin >> x;
    std::cout << "Enter nonnegative exponent n: ";
    std::cin >> n;

    if (n < 0) {
        std::cout << "n must be nonnegative." << std::endl;
        return 1;
    }
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    std::cout << "rpower("  << x << ", " << n << ") = " << rpower(x, n)  << std::endl;
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to compute = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    std::chrono::steady_clock::time_point begin2 = std::chrono::steady_clock::now();
    std::cout << "brpower(" << x << ", " << n << ") = " << brpower(x, n) << std::endl;
    std::chrono::steady_clock::time_point end2 = std::chrono::steady_clock::now();

    std::cout << "Time taken to compute = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end2 - begin2).count()
              << " microseconds" << std::endl;
    return 0;
}
```

OUTPUT

```
Enter base x:4

Enter nonnegative exponent n:9

rpower(4, 9) = 262144
Time taken to compute = 1437 microseconds
brpower(4, 9) = 262144
Time taken to compute = 417 microseconds
```

CONCLUSION

- GALVEZ
  This activity taught me how theoretical analysis differs from experimental analysis. After evaluation runtime using total comparisons, we tested how hardware and user inputs affect real computation speed. Our experimental results matched our theoretical predictions.

- PASION
  In this activity, I learned how to differentiate theoretical analysis and experimental analysis in coding algorithms. For the procedure, we first delved into how to read an algorithm theoretically, without the factor of hardware speed. We identified the total number of comparisons that would reflect the actual program's output. Then we moved onto experimental demonstrations. This is where we saw how hardware and the input of the user is a huge factor that affects the computation time. Lastly for the supplementary, this is where we demonstrated our theoretical and experimental analysis through the given algorithms. For both of the problem's analyses, the time complexity and actual output turned out to be aligned with each other. I'd say this was a good activity to learn about theoretical and experimental analysis. I just do need to improve on analyzing faster and clearer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.